

Laporan Tugas Kecil 1 IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Penyelesaian Permainan *Queens* LinkedIn

Arina Azka and 13524049¹

¹13524049@std.stei.itb.ac.id, ²13524049@gmail.com

Queens LinkedIn adalah permainan logika yang tersedia pada situs LinkedIn. Tujuan utama permainan ini adalah menempatkan tepat satu *Queen* pada tiap baris, kolom, dan daerah warna di dalam sebuah papan persegi berwarna.

Permainan *Queen LinkedIn* dapat dikatakan sebagai pengembangan dari persoalan klasik *N Queen Problem*. Meskipun memiliki dasar yang serupa, *Queens* LinkedIn menerapkan beberapa aturan tambahan berupa pembagian wilayah warna dan aturan ketentanggaan yang spesifik. Berikut ini adalah aturan permainan *Queens* LinkedIn.

- Hanya terdapat tepat satu *Queen* pada setiap baris.
- Hanya terdapat tepat satu *Queen* pada setiap kolom.
- Hanya terdapat tepat satu *Queen* pada setiap wilayah warna.
- Beberapa *Queen* dapat terletak pada garis diagonal yang sama selama posisinya tidak tepat bersebelahan.

Laporan kali ini menyajikan pendekatan komputasi untuk menyelesaikan permainan *Queens* LinkedIn menggunakan algoritma brute force murni yang dituliskan dalam bahasa pemrograman Python.

Algoritma Brute Force

Algoritma brute force merupakan algoritma sederhana yang menyelesaikan permasalahan menggunakan strategi lampang atau naif. Terdapat dua pendekatan Algoritma brute force yang diterapkan pada laporan ini, yaitu *exhaustive search* dan *backtracking*.

a. *Exhaustive search*

Algoritma brute force pertama yang diterapkan dalam laporan ini diklasifikasikan sebagai algoritma brute force murni atau *exhaustive search*. *Exhaustive search* didefinisikan sebagai metode pencarian sistematis yang mengeksplorasi setiap opsi hingga jawaban suatu masalah ditemukan. Pendekatan ini menjadi algoritma *default* yang diterapkan pada aplikasi ketika pengguna menonaktifkan toggle “Enable Optimization”.

Exhaustive search umumnya diterapkan dengan membuat semua opsi, lalu memeriksa apakah opsi tersebut adalah solusi yang valid atau bukan. Dengan mempertimbangkan jumlah opsi yang sangat besar dan tumbuh secara eksponensial, penulis melakukan sedikit modifikasi. Pada penerapannya, aplikasi memeriksa setiap opsi, yaitu sebuah papan utuh dengan seluruh *Queen* diletakkan, tepat setelah opsi tersebut dibuat menggunakan proses rekursi. Meskipun pengecekan per kolom pada sebuah papan dihentikan tepat setelah papan tersebut dikonfirmasi salah, pendekatan ini tetap dikategorikan sebagai *exhaustive search*. Hal ini dikarenakan tidak adanya mekanisme *pruning* (pemangkasan) di tengah proses pembentukan opsi, sehingga algoritma tetap dipaksa mengunjungi setiap cabang hingga kedalaman maksimal.

b. *Backtracking*

Berbeda dengan *exhaustive search*, pendekatan *backtracking* pada aplikasi ini mengintegrasikan mekanisme *pruning* atau pemangkasan ruang pencarian. Dengan mengacu pada algoritma sudoku yang terdapat pada halaman 45 PPT Brute-Force Bagian.1 2025/2026 ([https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-(2026)-Bag1.pdf)), optimasi dilakukan dengan cara memvalidasi setiap langkah penempatan *Queen* secara *real-time* di setiap level rekursi

Backtracking adalah algoritma pemecahan masalah yang melibatkan pencarian solusi secara bertahap dengan mencoba berbagai pilihan dan membatalkannya jika pilihan tersebut mengarah ke jalan buntu. Pendekatan ini menjadi opsi algoritma kedua dan dapat digunakan pengguna ketika mengaktifkan toggle “Enable Optimization”.

Jika pada suatu langkah penempatan *Queen* ditemukan pelanggaran aturan, seperti pelanggaran aturan baris, wilayah warna, atau posisi tetangga, maka algoritma akan segera menghentikan penelusuran pada cabang tersebut tanpa harus menunggu papan terisi penuh. Proses ‘mundur’ atau *backtrack* ini dilakukan untuk mencoba kemungkinan angka atau posisi lain pada kolom sebelumnya. Dengan membuang ribuan hingga jutaan jalur yang sudah pasti tidak mengarah pada solusi, *backtracking* secara drastis mengurangi total kasus yang perlu diperiksa. Oleh karena itu, pendekatan ini menawarkan waktu eksekusi yang jauh lebih efisien bagi pengguna, terutama pada ukuran papan yang kompleks.

Analisis dan Rancangan

a. Batasan Masalah

Untuk memastikan aplikasi berjalan sesuai dengan aturan permainan *Queens LinkedIn*, ditetapkan batasan-batasan berikut:

- **Struktur Papan**

Papan harus berbentuk persegi dengan n jumlah baris sama dan n jumlah kolom.

- **Daerah Warna**

Setiap papan terdiri dari n wilayah warna yang berbeda, ditandai dengan satu huruf kapital (A-Z).

b. Representasi Persoalan

Persoalan dimodelkan menggunakan beberapa struktur data utama dalam bahasa pemrograman Python, antara lain:

- **Matriks Papan (matrix)**

List dua dimensi yang menyimpan karakter huruf untuk merepresentasikan pembagian wilayah warna pada papan.

- **Array Solusi (current_p)**

List satu dimensi berukuran n dengan indeks list merepresentasikan nomor kolom, dan nilai di dalamnya merepresentasikan nomor baris tempat *Queen* diletakkan.

- **Pemetaan Warna (color_map)**

Dictionary yang memetakan setiap karakter wilayah ke nilai warna RGB untuk visualisasi gambar.

c. Inisiasi

Proses inisiasi merupakan langkah awal untuk menyiapkan data sebelum algoritma pencarian dijalankan.

1) Input Data

Program menerima masukan berupa berkas .txt yang berisi matriks karakter kapital (A-Z) yang merepresentasikan wilayah warna pada papan.

2) Validasi Struktur

Memeriksa apakah matriks tersebut berbentuk persegi. Jika jumlah baris tidak sama dengan jumlah kolom, jumlah warna tidak sama dengan ukuran papan, atau terdapat karakter yang tidak valid, program akan menampilkan pesan kesalahan.

3) Penentuan Skala

Menghitung nilai n (ukuran papan) yang juga menentukan jumlah *Queen* yang harus diletakkan serta jumlah wilayah warna yang harus unik.

4) Konfigurasi Visual

Menyiapkan *color_map* untuk memetakan setiap huruf wilayah ke warna RGB tertentu guna keperluan *rendering* pada antarmuka grafis.

5) **Penyiapan Wadah Solusi**

Menginisialisasi list kosong *current_p* (yang akan menampung posisi *Queen* selama proses pencarian berlangsung).

d. **Prosedur *Exhaustive search***

Pendekatan ini berfokus pada pembangkitan opsi susunan papan sebelum melakukan validasi akhir. Prosedur ini diterapkan melalui fungsi *recursiveWithoutPruning* dengan mekanisme sebagai berikut.

- 1) Mulai dari kolom pertama dan coba letakkan *Queen* pada setiap baris (0 hingga $n-1$) tanpa melakukan pengecekan aturan di tengah proses.
- 2) Lanjutkan proses penempatan *Queen* ke kolom berikutnya secara rekursif hingga seluruh kolom (n) terisi oleh *Queen*.
- 3) Setiap kali satu susunan papan utuh terbentuk, lakukan pengecekan validitas secara menyeluruh menggunakan fungsi *isSolution*.
- 4) Lakukan pengecekan untuk memastikan susunan memenuhi seluruh aturan yang telah ditetapkan.
- 5) Jika susunan tersebut valid, maka solusi ditemukan dan proses dihentikan.
- 6) Jika susunan tidak valid, abaikan dan lanjutkan untuk membangkitkan kombinasi angka baris berikutnya pada kolom terakhir.

e. **Prosedur *Backtracking***

Pendekatan ini mengoptimalkan pencarian dengan melakukan validasi di setiap langkah penempatan *Queen*.

1. Mulai dari kolom pertama dan coba letakkan *Queen* pada baris tertentu di kolom tersebut.
2. Untuk setiap penempatan, lakukan validasi *real-time* untuk memastikan posisi tersebut memenuhi aturan.
3. Jika posisi ditemukan aman, letakkan *Queen* dan segera berlanjut ke kolom berikutnya untuk mencari posisi *Queen* selanjutnya.
4. Jika posisi tidak aman, coba letakkan *Queen* pada baris berikutnya di kolom yang sama.
5. Jika tidak ditemukan baris yang aman hingga baris terakhir di kolom tersebut, lakukan *backtracking*, yaitu hapus *Queen* di kolom saat ini dan kembali ke kolom sebelumnya untuk mencoba baris selanjutnya di kolom tersebut.

f. Rancangan GUI

Antarmuka pengguna dibangun menggunakan pustaka `customtkinter` dengan pembagian *section* sebagai berikut.

- **Control Section**

Terdiri dari tombol *Import File* untuk masukan data `.txt`, *toggle* "Enable Optimization" untuk memilih algoritma, serta tombol *Solve*, *Save*, dan *Reset*.

- **Output Section**

- **Frame Visualisasi**

Merender matriks papan menjadi gambar berwarna dan menampilkan posisi *Queen* secara *real-time*.

- **Frame Statistik**

Menampilkan kinerja aplikasi berupa *Execution Time* dan *Total Cases Checked* yang diperbarui secara *real-time*.

Implementasi

Akibat panjangnya implementasi kode untuk menyelesaikan permainan ini, penulis tidak dapat menyertakan seluruh kode dalam laporan. Sebagai alternatif, penulis memaparkan penjelasan fungsi-fungsi yang digunakan dengan mengelompokkannya ke dalam beberapa modul berikut. Fungsi-fungsi tersebut dikelompokkan dalam beberapa modul sebagai berikut.

a. *Queens_linked.py*

Modul ini berisi algoritma inti dari aplikasi karena menangani logika utama untuk menyelesaikan permainan *Queens* LinkedIn menggunakan dua pendekatan, yaitu *Backtracking* dan *Exhaustive search*. Modul ini bertanggung jawab untuk memvalidasi papan, memeriksa aturan penempatan *Queen*, serta menyediakan mekanisme *callback* untuk visualisasi proses pencarian secara *real-time*.

1) **isValid(matrix)**

Fungsi ini adalah validasi awal untuk memastikan papan memenuhi syarat teknis permainan, yaitu papan tidak kosong, papan berbentuk persegi, setiap elemen adalah huruf kapital (representasi warna), dan jumlah warna unik harus tepat sama dengan ukuran papan.

2) **isSafe(current_p, new_row, colored_board)**

Fungsi menentukan apakah *Queen* baru bisa diletakkan di baris tertentu pada kolom saat ini tanpa melanggar aturan. Beberapa aturan yang diperiksa adalah tidak ada *Queen* di baris yang sama, tidak ada *Queen* di wilayah warna yang sama, dan *Queen*

tidak bersinggungan langsung secara vertikal atau diagonal dengan *Queen* di kolom sebelumnya.

3) isSolution(p, colored_board)

Fungsi yang digunakan oleh metode *exhaustive search* untuk mengecek apakah susunan *Queen* yang sudah lengkap memenuhi semua kriteria keamanan.

4) recursiveWithPruning(n, current_p, colored_board)

Fungsi implementasi algoritma *backtracking* yang mencoba meletakkan *Queen* kolom demi kolom. Jika di tengah jalan ditemukan posisi yang tidak aman (isSafe bernilai False), algoritma akan langsung berhenti di cabang tersebut (*pruning*) dan mencoba kemungkinan lain. Fungsi ini juga memberikan visual_callback untuk memperbarui tampilan GUI.

5) recursiveWithoutPruning(n, current_p, colored_board)

Fungsi implementasi algoritma *exhaustive search* yang mencoba semua kemungkinan posisi *Queen* tanpa memedulikan aturan di tengah proses. Aturan hanya dicek menggunakan isSolution setelah semua *Queen* diletakkan.

6) findFirstSolution(n, colored_board, use_pruning=True)

Fungsi untuk menginisialisasi penghitung kasus ($i = 0$) dan memilih metode pencarian berdasarkan parameter use_pruning yang dipilih pengguna.

b. file_processing.py

Modul file_processing.py bertanggung jawab atas seluruh operasi Input/Output (I/O) yang berkaitan dengan manajemen berkas teks dalam aplikasi.

1) readFile(filename)

Untuk mengonversi data dari berkas teks mentah menjadi matriks yang dapat diolah oleh algoritma.

2) matrixToImage(matrix, solution, save_path, color_map)

Bertugas membuat gambar papan permainan berdasarkan skema warna dan posisi *Queen*. Fungsi ini menggunakan color_map untuk memberikan warna unik pada setiap daerah warna di papan yang diwakili oleh huruf A-Z.

c. main.py

Modul main_gui.py merupakan antarmuka grafis utama dari aplikasi *Queens* LinkedIn. Modul ini dibangun menggunakan pustaka customtkinter. Peran utamanya adalah sebagai pengendali yang menghubungkan interaksi pengguna dengan logika pemrosesan data dan algoritma pencarian. Modul ini juga menerapkan konsep multithreading agar antarmuka tetap responsif saat algoritma sedang bekerja.

1) **importFile()**

Bertugas menangani proses pengunggahan berkas konfigurasi papan (.txt). Apabila berkas berhasil dibuka, fungsi ini memperbarui status tombol "Solve", "Save", dan "Reset" menjadi aktif.

2) **solveHandler() dan solveGame()**

Keduanya berperan dalam memulai proses pencarian solusi. SolveHandler() membuat thread baru agar proses pencarian tidak membekukan aplikasi, sedangkan solveGame() memicu fungsi findFirstSolution(), mengatur mekanisme monitoring iterasi, dan menangani perubahan status label status permainan.

3) **updateBoardDisplay(solution, is_preview)**

Bertugas menampilkan gambar papan secara dinamis ke area visualisasi. Gambar papan yang ditampilkan berupa pratinjau papan yang baru diunggah maupun menampilkan progres papan saat algoritma sedang berjalan hingga selesai.

4) **resetHandler(opt_switch)**

Apabila ditekan, fungsi ini mengembalikan aplikasi ke kondisi awal, yaitu tidak ada papan yang sudah diunggah, mematikan tombol-tombol fungsional, dan mengembalikan statistik kinerja aplikasi menjadi 0.

5) **saveResultHandler()**

Fungsi ini mampu mengunduh gambar papan yang ditampilkan pada area visualisasi ke dalam format .png, baik papan awal ataupun hasil solusi.

6) **Inisiasi GUI**

Bertugas menyiapkan kerangka utama aplikasi, mengatur tema, dan resolusi jendela

7) **Fungsi Pembangun Antarmuka**

Fungsi-fungsi ini digunakan untuk menyusun tata letak (layout) aplikasi secara modular.

a) **header(main_frame)**

Menampilkan judul utama dan identitas pembuat (Arina Azka - 13524049).

b) **controlSection(main_frame)**

Membangun panel sisi kanan yang berisi tombol Import, toggle optimasi (Enable Optimization), tombol Solve, Save, dan Reset.

c) **outputSection(main_frame)**

Wadah untuk bagian statistik dan visualisasi di sisi kiri.

- **visualSection(output_frame)**

Menyiapkan area bingkai tempat gambar papan akan ditampilkan.

- **statsSection(output_frame)**

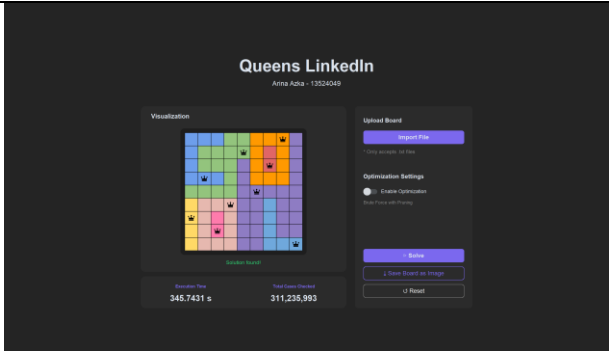
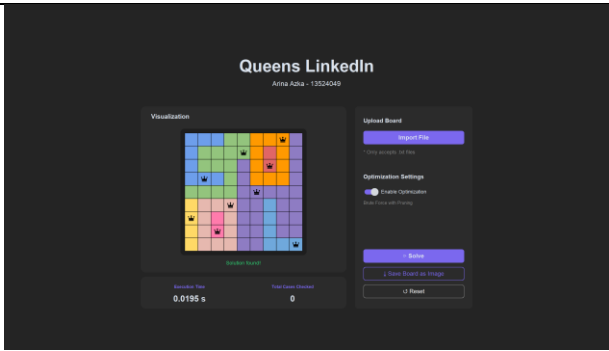
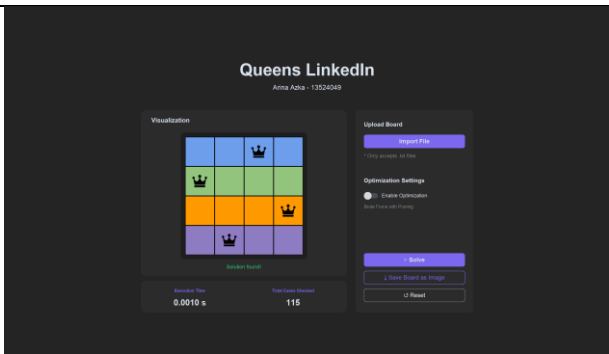
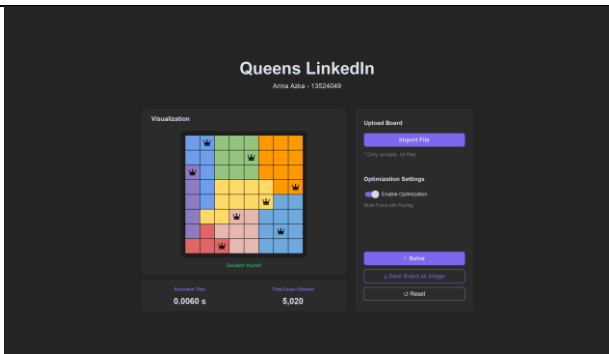
Menyiapkan label untuk menampilkan "Execution Time" dan "Total Cases Checked".

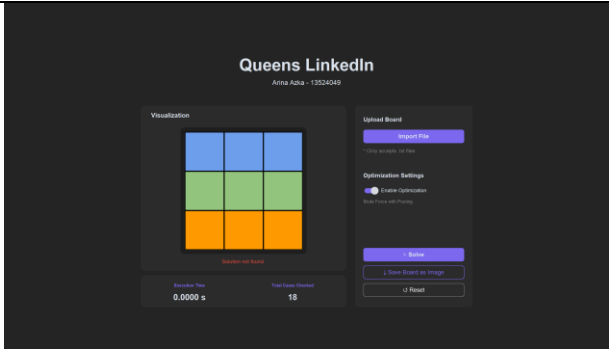
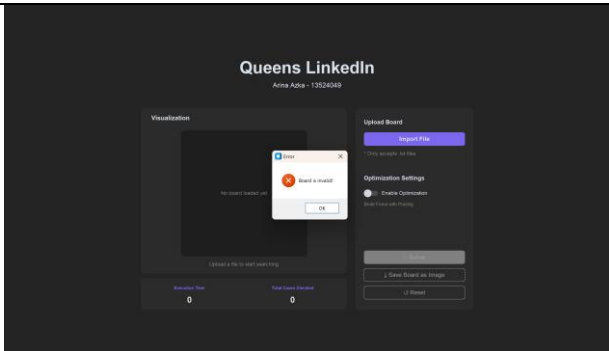
d) mainApp()

Berfungsi untuk dan menggunakan header(), contentSection(), dan main_frame untuk menyusun tata letak. Fungsi pengikat akhir yang menyusun seluruh bagian (header dan content) ke dalam kontainer pusat aplikasi.

Tangkapan Layar

Input	Output	Keterangan
-		Tampilan awal ketika aplikasi dibuka, belum ada input yang dimasukkan
AAABBCCCD ABBBBCECD ABBBDCEDD AAABDCCCD BBBBDDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika mengunggah file input berisi board yang valid (.txt).
AAABBCCCD ABBBBCECD ABBBDCEDD AAABDCCCD BBBBDDDDDD FGGGDDHDD FGIGDDHDD		Tampilan ketika aplikasi sedang melakukan pencarian tanpa optimasi.

FGIGDDHDD FGGGDDHHH		
AAABBCCCD ABBBBCECD ABBBDCEDC AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika aplikais berhasil menemukan solusi untuk tc1 dengan perncarian yang tidak dioptimasi.
AAABBCCCD ABBBBCECD ABBBDCEDC AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika aplikasi berhasil menemukan solusi untuk tc1 dengan perncarian yang dioptimasi.
AAAA BBBB CCCC DDDD		Tampilan ketika aplikasi berhasil menemukan solusi untuk tc2 dengan perncarian yang tidak dioptimasi.
AABBBCCC AABBBCCC DABBBCCC DAFFFFCC DAFFFFHH DFFGGHHH		Tampilan ketika aplikasi berhasil menemukan solusi untuk tc3 dengan perncarian yang dioptimasi.

DEGGGHHH EEEGGHHH		
AAA BBB CCC		Tampilan ketika aplikasi gagal menemukan solusi untuk tc4 dengan pencarian yang dioptimasi.
AAABBCCCD ABBBBCECDA ABBBDCEDD AAABDCCCD BBBBDDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH		Tampilan ketika mengunggah file input (tc5) berisi board yang tidak valid.

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Arina Azka

Appendiks

https://github.com/arinazk/Tucil1_13524049

Referensi

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/02-Algoritma-Brute-Force-(2026)-Bag1.pdf)

<https://www.joelotz.com/blog/2020/n-Queens-puzzle-part-2-brute-force-algorithm.html>
python-pillow.github.io

<https://www.geeksforgeeks.org/python/convert-a-numpy-array-to-an-image/>

<https://www.pythoninformer.com/python-libraries/numpy/numpy-and-images/>

<https://medium.com/@whyamit101/step-by-step-guide-to-convert-numpy-array-to-pil-image-f7f8492cd785>

<https://customtkinter.tomschimansky.com/>

https://www.youtube.com/watch?v=Miydkti_QVE

<https://youtu.be/aRJXC8hJvrc?si=y6JAcKiWBecshqsb>

Lampiran

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainannya	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	